Classification du genre (mouvement) d'oeuvres d'arts

Sebastian Caron Lina Lethoor Gregory Duporge Julien Biout

2 Novembre 2026

Ce projet consiste à classifier le style artistique d'oeuvres d'art à travers leurs représentations latentes, intégrant un serveur MCP (Model Context Protocol).

Le modèle final peut classifier les oeuvres d'art dans 21 mouvements artistiques majeurs (Renaissance, Baroque, Impressionnisme, Cubisme, etc.) et extraire des embeddings pour la recherche d'analogies visuelles.

I. Installation

1. Installation de l'environnement

Prérequis

- Python 3.10+ (testé avec Python 3.12 / 3.14)
- Carte graphique NVIDIA recommandée avec pilotes CUDA à jour

Création du venv

```
# Cloner le dépôt et se placer dans le projet
cd project

# Créer et activer l'environnement virtuel
python3 -m venv .venv
source .venv/bin/activate
```

Installation des dépendances

```
# Dépendances de base (PyTorch, Torchvision, Pillow)
pip install -r requirements.txt

# Si vous disposez d'un GPU NVIDIA (CUDA 12.1+), installez la version GPU de
↪ PyTorch :
# pip install torch torchvision --index-url
↪ https://download.pytorch.org/whl/cu121

# Dépendances pour le serveur MCP et les outils d'indexation
pip install "mcp>=2.3.0" tqdm
```

---

2. Téléchargement & Préparation du dataset (ART500K)

Le projet utilise le dataset **ART500K** (~500 000 images d'oeuvres d'art et leurs métadonnées).

> **Note :** Choisissez "Raw images of visual arts with general label list: Data&Labels(56.6GB)/Labels".

Structure attendue dans `data/`

Après extraction, le dossier `data/` doit avoir l'arborescence suivante :

```
data/
├── Art Movement/   # 119 dossiers de mouvements
├── Artists1/       # Oeuvres par artistes (partie 1)
├── Artists2/       # Oeuvres par artistes (partie 2)
├── Medium/         # Oeuvres classées par support
├── label_list.csv  # Table de métadonnées
├── head_info.csv   # En-têtes des métadonnées
└── Readme
```

Extraction de l'archive

Si vous disposez de l'archive `art500k.tar.gz` (téléchargement ou support externe) :

```
mkdir -p data
tar -xzf art500k.tar.gz -C data/
```

> **Note :** Le chargeur de données (`src/loader.py`) filtre et associe automatiquement les sous-dossiers de `data/Art Movement/` aux 21 classes canoniques sans conflit d'étiquetage.
>
> **Note encore :** Nous n'utilisons pas le dossier Artists1 car il n'y a quasiment pas d'informations sur le genre. Nous aurions pu cependant intégrer Artists2 mais cela ajoute des problèmatiques d'équilibrage et représentation des classes, en plus d'allonger le temps d'entrainement. Nous nous reservons le droit d'intégrer les données, de poursuivre l'entrainement et d'améliorer les models pour la V2.

II. Méthodologie

1. Structure du projet

Le projet est conçu de façon modulaire afin de découpler la préparation des données, l'entraînement des modèles de vision, l'analyse expérimentale (ablation, validation croisée, métriques) et l'interfaçage avec un agent conversationnel via le protocole MCP (*Model Context Protocol*).

```
project/
├── data/           # Données brutes ART500K
├── checkpoints/    # Poids et checkpoints
│   ├── gallery_index.pt # Embeddings 512D
│   ├── ablation/   # Métriques étude d'ablation
│   ├── cross_val/  # Validation croisée 3 plis
│   └── ArtClassifier/ # Modèles entrainés
├── rapports/       # Rapports et figures
│   ├── confusion_matrix_canonique.png
│   ├── confusion_matrix_non_canonique.png
```

```
|   ├── november-1.md # Rapport intermédiaire
|   └── november-2.md # Rapport final
├── src/            # Code source modulaire
|   ├── models/     # CustomArtCNN, ConvNeXt
|   ├── serv_mcp/   # Serveur MCP et outils LLM
|   ├── loader.py   # Dataset et taxonomie
|   ├── early_stopping.py # Arrêt anticipé
|   ├── trainer.py  # Entraînement AMP fp16
|   ├── ablation.py # Étude d'ablation
|   ├── cross_validation.py # CV 3 plis
|   └── measures.py # Métriques complètes
├── requirements.txt # Dépendances
└── README.md       # Guide d'exécution
```

Rôles des principaux composants logiciels

1. **`src/loader.py`** :
   - Effectue la résolution et le regroupement taxonomique des 119 dossiers initiaux en **21 classes canoniques** mutuellement exclusives via le dictionnaire `TAXONOMY_MAPPING`.
   - Construit les pipelines de transformation avec `torchvision.transforms.v2` (augmentations riches en entraînement, redimensionnement bicubique et recadrage centré en inférence).
2. **`src/models/`** :
   - `CustomArtCNN` : CNN classique à 4 blocs convolutionnels avec normalisation par lots, servant de baseline entraînée à partir de zéro (*from scratch*).
   - `ArtClassifierTinyBased` : Modèle de production s'appuyant sur un backbone `convnext_tiny` préentraîné, projeté dans un espace latent à 512 dimensions avant la tête de classification.
3. **`src/serv_mcp/`** :
   - Implémente le protocole MCP (*Model Context Protocol*) permettant à des LLM (comme Claude, Antigravity ou Gemini) d'invoquer le modèle de vision en tant qu'outil outillé (*tool use*).
4. **`src/trainer.py` & `src/early_stopping.py`** :
   - Assurent un entraînement robuste avec précision mixte (`torch.autocast(fp16)`), suivi en temps réel de la loss et des métriques Micro/Macro Top-1 et Top-5, ainsi que l'arrêt anticipé avec restauration automatique du meilleur checkpoint.
5. **`src/ablation.py` & `src/cross_validation.py`** :
   - Fournissent les protocoles expérimentaux d'analyse fine (isolation des composants du réseau, mesure de la variance inter-plis sur 16 612 images).

---

2. Intégration LLM & Protocole MCP

Architecture globale de l'interaction LLM-Vision

Pour exploiter notre modèle spécialiste dans un cadre interactif, nous avons intégré le protocole standardisé **MCP (Model Context Protocol)**. Ce protocole résout une limitation majeure des LLM textuels : leur incapacité à percevoir directement les caractéristiques plastiques, les textures picturales et les relations de proximité dans l'espace des styles artistiques.

```
[Utilisateur] <---> [Agent LLM]
                          | (stdio / JSON-RPC)
                          v
                [Serveur ArtExplorerMCP]
                /         |          \
       (Inférence) (Cosinus 512D) (Métadonnées)
            v           v            v
     [ConvNeXt-Tiny] [Index.pt]   [label_list.csv]
```

Modèle de langage (LLM) et Prompt Système

Le système a été validé avec des agents LLM avancés (Claude 3.5 Sonnet / Antigravity / Gemini) supportant nativement l'appel d'outils (*Tool Calling*). Pour orienter le LLM vers une posture d'expert en histoire de l'art assisté par vision artificielle, le prompt système suivant a été conçu :

```
Tu es un historien de l'art assisté par une IA spécialisée en vision par
↪ ordinateur.
Tu disposes d'un modèle de Deep Learning spécialiste (ArtClassifierTinyBased
↪ basé sur ConvNeXt-Tiny)
connecté via des outils MCP.

Tes objectifs :
1. Lorsqu'un utilisateur te soumet une œuvre d'art (via son chemin de
↪ fichier, URL ou identifiant),
   utilise systématiquement l'outil `predict_style` pour obtenir les
↪ prédictions du modèle spécialiste.
2. Interprète les probabilités retournées : analyse la confiance du Top-1 et
↪ examine les styles concurrents
   dans le Top-5 (ex. une confusion entre Baroque et Maniérisme témoigne
↪ souvent d'une période de transition).
3. Si l'utilisateur souhaite explorer des analogies visuelles ou des
↪ inspirations, utilise `find_similar_artworks`.
4. Pour analyser les influences croisées entre mouvements distincts, utilise
↪ `find_cross_style_neighbors`.
5. Fournis toujours une analyse historique et esthétique détaillée, en
↪ articulant les métadonnées de l'œuvre
   (artiste, date, mouvement réel) avec les observations plastiques
↪ (composition, lumière, touche, palette).
```

Outils exposés via le serveur MCP

Le serveur MCP (`src/serv_mcp/mcp_server.py`) communique via le transport standardisé `stdio` et expose 4 outils principaux :

1. **`predict_style(image_path_or_id)`** :
   - Reçoit un chemin local, une URL ou un identifiant d'œuvre.
   - Charge et prétraite l'image en 288×288 selon la norme bicubique ImageNet.
   - Extrait le vecteur latent 512D et calcule les logits de classification.
   - Retourne un dictionnaire JSON structuré contenant : le style prédit, l'indice de confiance (0 à 1), la distribution de probabilité du Top-5, ainsi que les métadonnées officielles issues de `label_list.csv` (titre, peintre, style catalogué, concordance `is_correct`).
2. **`find_similar_artworks(image_path_or_id, k=5)`** :
   - Extrait l'embedding 512D de l'image requête.
   - Calcule la similarité cosinus vectorisée avec les 3 313 représentations latentes de la galerie pré-

indexée :

$$\mathrm{Sim}(u, v) = \frac{u \cdot v}{\|u\|_2 \, \|v\|_2}$$

- Retourne les k œuvres les plus proches avec leur score de similarité et leurs métadonnées complètes.

3. **`find_cross_style_neighbors(image_path_or_id, k=5)`** :
   - Similaire à `find_similar_artworks`, mais applique un masque sur les œuvres appartenant au même mouvement artistique que l'image requête.
   - Permet à l'historien ou à l'amateur d'explorer des ponts visuels et des continuités stylistiques par-delà les frontières temporelles ou géographiques (ex. similitude de palette entre un clair-obscur baroque et une scène impressionniste nocturne).
4. **`get_model_info()`** :
   - Retourne l'architecture sous-jacente (`ConvNeXt-Tiny`), le volume d'œuvres référencées en galerie, la dimension de l'espace latent (512) et la liste exhaustive des 21 classes canoniques.

---

3. Modèle Spécialiste & Protocole Expérimental

3.1 Architecture du Modèle Spécialiste (`ArtClassifierTinyBased`)

Le modèle sélectionné pour le déploiement est **`ArtClassifierTinyBased`**, implémenté dans `src/models/pretrained.py`.

Il se compose de : 1. **Backbone ConvNeXt-Tiny** : - Modèle convolutionnel moderne incorporant les avancées des Vision Transformers (convolutions séparables en profondeur 7×7, normalisation `LayerNorm`, activations `GELU`, expansion inversée $1 \times 1 \to 7 \times 7 \to 1 \times 1$). - Pré-entraîné sur ImageNet-1K (`ConvNeXt_Tiny_Weights.DEFAULT`), fournissant une extraction robuste de descripteurs visuels de bas et moyen niveau (bords, textures, contrastes, compositions). - Dimension de sortie du pooling adaptatif : **768 dimensions**. 2. **Tête de Projection Latente (Espace d'Embedding)** : - Une couche linéaire dense : $\mathbb{R}^{768} \to \mathbb{R}^{512}$. - Une normalisation par lots `BatchNorm1d(512)`. - Une fonction d'activation non linéaire `GELU()`. - Une couche de régularisation par masquage aléatoire `Dropout(p=0.4)`. 3. **Tête de Classification** : - Une couche linéaire finale : $\mathbb{R}^{512} \to \mathbb{R}^{21}$ projetant l'embedding vers les scores de log-vraisemblance des 21 mouvements artistiques. - Initialisation de Kaiming normale (`kaiming_normal_`) sur les poids de la projection et du classifieur avec biais initialisés à zéro.

3.2 Justification des Hyperparamètres

Le choix des hyperparamètres a été guidé à la fois par des fondements théoriques en vision par ordinateur et par les contraintes matérielles réelles de notre banc d'essai (GPU NVIDIA RTX 3050 Laptop / 4 Go de VRAM).

3.3 Choix des Classes & Canonisation Taxonomique

Le jeu de données brut **ART500K** organise initialement ses dossiers en plus de 117 sous-dossiers très fragmentés, présentant des redondances lexicales et historiques majeures : - Des subdivisions régionales d'un même courant : *Florentine painting*, *Venetian school*, *Bolognese School*, *Northern Renaissance* coexistaient avec *Renaissance*. - Des catégories génériques englobantes : *Modern art* et *Modernism*. - Des dénominations doublons : *Social realism* et *Socialist realism*.

Pour remédier à ces confusions structurelles qui empêchent la convergence d'un classifieur probabiliste, nous avons mis en place une table de correspondance canonique (`TAXONOMY_MAPPING` dans `src/loader.py`) regroupant l'ensemble des œuvres en **21 mouvements artistiques majeurs et cohérents**.

Validation empirique : Comparaison des matrices de confusion Pour attester rigoureusement de la pertinence de ce regroupement canonique, deux modèles identiques ont été entraînés et évalués sur la totalité de leurs classes respectives *(les représentations graphiques haute résolution des deux matrices sont détaillées en Annexe A)* :

1. **Modèle Canonique (21 classes)** — Figure haute résolution :
   - **Micro Top-1 : 81,20%** sur 3 313 images de validation.
   - La diagonale principale est très fortement marquée : *Street Art & Art Public* culmine à **96%**, *Réalisme Social* à **100%**, *Art Japonais* à **90%**, *Expressionnisme* à **91%**, *Impressionnisme* à **84%**.
   - Les quelques confusions résiduelles sont **historiquement et artistiquement fondées** : par exemple, le *Maniérisme* est confondu à 40% avec la *Renaissance* (dont il constitue l'évolution stylistique directe), et le *Rococo* est classé à 15% en *Baroque*.
2. **Modèle Non Canonique (117 classes brutes)** — Figure haute résolution :
   - **Micro Top-1 : 65,67%** sur 4 087 images de validation (chute de **-15,5%**).
   - La matrice présente une forte dispersion hors diagonale avec des bandes verticales prononcées : les sous-classes locales sont systématiquement

Hyperparamètre	Valeur retenue	Justification théorique et expérimentale
Résolution image (`img_size`)	288×288	Supérieure à la résolution standard 224×224. Dans l'analyse artistique, la texture des toiles, la finesse des coups de pinceau et les craquelures requièrent un niveau de détail élevé. Le ratio d'évaluation utilise un redimensionnement bicubique à 329 pixels suivi d'un `CenterCrop(288)`.
Optimiseur	Adam	Permet un calcul de taux d'apprentissage adaptatif par paramètre via l'estimation des moments d'ordre 1 et 2. Crucial ici car le réseau couple un backbone pré-entraîné (nécessitant des mises à jour douces) avec une tête latente nouvellement initialisée.
Taux d'apprentissage (η)	1×10^{-4}	Compromis optimal pour le fine-tuning : un learning rate de 10^{-3} déstabiliserait les filtres pré-entraînés de ConvNeXt, tandis qu'un taux de 10^{-5} ralentirait excessivement la convergence de la tête latente.
Taille de batch (B)	16	Contrainte matérielle stricte : un batch de 32 en résolution 288×288 provoque un dépassement de mémoire (*CUDA Out-Of-Memory*) sur 4 Go de VRAM. Un batch de 16 offre une variance de gradient suffisamment stable tout en tenant dans l'enveloppe mémoire.
Précision mixte (AMP)	`torch.autocast(fp16)` + `GradScaler`	Divise par deux l'empreinte mémoire des activations et accélère le calcul tensoriel sur les Tensor Cores, sans dégradation de la précision grâce au `GradScaler` qui prévient le sous-dépassement numérique (*underflow*).
Dimension latente	512	Goulot d'étranglement dimensionnel compact servant à la recherche vectorielle dans le serveur MCP. Assez vaste pour coder les nuances stylistiques, mais suffisamment réduit pour garantir des recherches par similarité cosinus quasi-instantanées (< 2 ms).
Dropout (p)	$0,4$	Préserve le modèle du surapprentissage dans la tête de classification, particulièrement sur les classes minoritaires ayant peu d'exemples d'entraînement.
Activation	`GELU`	L'activation *Gaussian Error Linear Unit* offre une courbure lisse et différentiable, limitant le phénomène de neurones morts propre à ReLU et assurant une cohérence avec l'architecture native de ConvNeXt.
Arrêt anticipé (*Early Stopping*)	Patience de 2 époques, $\Delta_{\min} = 10^{-4}$	Surveille la perte de validation (`mode="min"`). Évite tout surapprentissage résiduel et restaure automatiquement le checkpoint au point d'inflexion optimal (`best_loss_model.pth`).
Augmentation de données	`transforms.v2` (Recadrage, Flip, Rotation, Jitter, Erasing)	`RandomResizedCrop(scale=(0.7, 1.0))` force l'invariance d'échelle ; `RandomHorizontalFlip` préserve le style pictural ; `RandomRotation(15°)` et `ColorJitter(0.15)` simulent les conditions de prise de vue muséographique ; `RandomErasing(p=0.2)` empêche le réseau de sur-focaliser sur un détail isolé.

attirées par les classes mères surreprésentées (*Baroque*, *Renaissance*).

Cette analyse comparative démontre que la canonisation taxonomique est indispensable pour obtenir des frontières de décision fiables et exploitables en aval par le serveur MCP.

3.4 Comparaison entre les Architectures de Modèles

Trois familles de modèles ont été implémentées et comparées :

- **`CustomArtCNN`** : Bien que rapide et léger, l'entraînement à partir de zéro sur un corpus de 16 612 images est insuffisant pour apprendre des représentations visuelles abstraites et complexes ; le modèle stagne autour de 41% de précision.
- **`ArtClassifier` (ResNet-50)** : Le backbone ResNet-50 traditionnel fournit une bonne base de transfert (77,8%), mais ses convolutions 3×3 standard manquent de champ récepteur pour capturer la composition globale d'une toile.
- **`ArtClassifierTinyBased` (ConvNeXt-Tiny)** : Obtient les meilleures performances grâce à ses filtres 7×7 à convolution en profondeur (*depthwise*), son organisation moderne des couches et sa tête latente 512D. C'est ce modèle qui a été retenu pour l'ensemble des études ultérieures et le serveur MCP.

3.5 Comparaison avec les Modèles Baselines

Afin de mesurer la valeur ajoutée réelle de notre modèle spécialiste, nous le comparons à plusieurs niveaux de référence (baselines théoriques, statistiques et heuristiques) sur le dataset canonique de 16 612 images (21 classes) :

Importance de la distinction Micro Top-1 vs Macro Top-1 Le tableau ci-dessus met en lumière un enjeu fondamental de notre projet : le **déséquilibre de distribution des données**. - La baseline de la classe majoritaire montre qu'un prédicteur trivial prédisant systématiquement *Réalisme Social* obtiendrait un score Micro Top-1 trompeur de **48,35%**, tout en ayant une exactitude de 0% sur les 20 autres mouvements artistiques (d'où son Macro Top-1 de $4,76\%$). - Notre modèle atteint **81,59% en Micro Top-1** et **50,18% en Macro Top-1**, ce qui confirme qu'il parvient à discriminer efficacement des mouvements minoritaires (ex. *Surréalisme* avec 59 images ou *Cubisme* avec 61 images).

Modèle	Type d'architecture	Paramètres totaux	Précision Top-1 (Validation)	Top-5 (Validation)	Vitesse d'inférence (img/s)
`CustomArtCNN`	CNN 4 blocs convolutifs (*from scratch*)	~4,2 M	~41,5%	~68,2%	**185 img/s**
`ArtClassifier`	ResNet-50 pré-entraîné ImageNet	24,5 M	77,8%	93,6%	110 img/s
`ArtClassifierTinyBased`	**ConvNeXt-Tiny pré-entraîné ImageNet**	**28,2 M**	**81,59%**	**96,17%**	**145 img/s**

Type de Baseline	Méthode / Modèle	Micro Top-1	Macro Top-1	Top-5	Commentaire méthodologique
Hasard uniforme	Tirage aléatoire équiprobable ($1/K$)	4,76%	4,76%	23,81%	Performance théorique nulle sans aucune information a priori (1/21).
Tirage stratifié	Tirage aléatoire selon la distribution réelle ($\sum p_i^2$)	26,19%	4,76%	~52%	Respecte les proportions de classes mais sans perception visuelle.
Classe majoritaire	Prédire constamment *Réalisme Social*	48,35%	4,76%	48,35%	Biais d'évaluation : score Micro artificiellement élevé car la classe majoritaire représente 48,35% des images (8 032 / 16 612), mais **Macro Top-1 catastrophique (4,76%)**.
CNN from scratch	`CustomArtCNN` (initialisé aléatoirement)	~41,5%	~25,0%	~68,2%	Modèle convolutionnel sans transfert de connaissances ImageNet.
Backbone gelé	`ArtClassifierTinyBased` (Backbone gelé, tête seule)	79,60%	44,73%	94,69%	Exploitation directe des descripteurs ImageNet sans adaptation fine.
Modèle spécialiste final	**`ArtClassifierTinyBased` (Fine-tuning complet)**	**81,59%**	**50,18%**	**96,17%**	**Surclasse nettement l'ensemble des baselines sur les trois métriques.**

3.6 Validation Croisée Stratifiée (3 Plis)

Pour évaluer la reproductibilité de l'entraînement et vérifier que les performances ne résultent pas d'un découpage chanceux des données, une validation croisée stratifiée à 3 plis ($K = 3$) a été menée sur l'intégralité des 16 612 images :

Enseignements de la validation croisée

1. **Remarquable stabilité** : L'écart-type de la précision globale est inférieur à un demi-pourcent ($\sigma = 0,46\%$), et celui du Top-5 est quasiment nul ($\sigma = 0,07\%$). Cette régularité prouve la robustesse du modèle face aux variations d'échantillonnage.
2. **Efficacité de l'Early Stopping** : Sur tous les plis, la convergence s'opère entre la 5e et la 7e époque. L'interruption anticipée protège efficacement le réseau contre le surapprentissage sur les classes denses.

3.7 Tests par Ablation

Une étude d'ablation systématique a été exécutée afin de quantifier l'apport individuel de chaque choix d'architecture dans `ArtClassifierTinyBased` :

Analyse des résultats d'ablation :

- **Fine-tuning du backbone (`frozen_backbone` vs `baseline`)** : Geler le backbone réduit les paramètres entraînables de 98,6% et assure une performance respectable (79,60%), confirmant l'excellente transférabilité de ConvNeXt. Toutefois, le dégel des couches convolutionnelles apporte un gain net de **+2,0% en Top-1**, **+5,45% en Macro Top-1** et abaisse la loss de 0,8059 à 0,6323.
- **Rôle de l'espace latent intermédiaire (`no_latent_space`)** : La projection directe 768 → 21 obtient un score légèrement supérieur en classification pure (82,16%). Néanmoins, la conservation du goulot d'étranglement 512D (`baseline`) est indispensable pour notre projet afin d'assurer l'indexation compacte et la recherche par similarité cosinus via MCP.
- **Rôle du Dropout et de la Normalisation** : L'ablation du dropout améliore ponctuellement le Macro Top-1 (52,13%) mais induit une remontée rapide de la perte de validation dès la 6e époque. L'ablation de la BatchNorm accélère la convergence (5 époques).

III. Développement & Difficultés Rencontrées

Au cours de la réalisation de ce projet, plusieurs défis techniques majeurs ont dû être surmontés :

1. Contraintes strictes de mémoire GPU (VRAM de 4 Go)

L'entraînement de réseaux modernes sur des images haute résolution requiert d'importantes ressources mémoire. Sur notre carte NVIDIA RTX 3050 (4 Go de VRAM dédiée),

Pli	Taille Entraînement	Taille Validation	Micro Top-1 (Max)	Macro Top-1 (Max)	Top-5 (Max)	Meilleure Loss	Époque d'arrêt
Fold 1	11 074	5 538	80,10%	44,58%	95,47%	0,7209	5 (Early stop)
Fold 2	11 075	5 537	80,96%	47,03%	95,47%	0,6733	7 (Early stop)
Fold 3	11 075	5 537	80,26%	47,59%	95,34%	0,7134	7 (Early stop)
Moyenne $\pm\sigma$	—	—	**80,44% ± 0,46%**	**46,40% ± 1,60%**	**95,42% ± 0,07%**	**0,7025 ± 0,0256**	**6,33 ± 1,15**

Configuration	Description	Paramètres entraînés	Micro Top-1 (Max)	Macro Top-1 (Max)	Top-5 (Max)	Meilleure Loss	Époque d'arrêt	Durée
`baseline`	Modèle complet (ConvNeXt-Tiny + Latent 512 + BN + GELU + Dropout 0.4)	28 225 653	**81,59%**	50,18%	**96,17%**	0,6323	8	25m 28s
`no_batchnorm`	Suppression de la `BatchNorm1d` dans la tête latente	28 224 629	**81,77%**	49,67%	95,80%	**0,6045**	5 (Early stop)	15m 49s
`no_dropout`	Suppression du `Dropout` ($p = 0.0$)	28 225 653	81,32%	**52,13%**	95,89%	0,6734	6 (Early stop)	19m 05s
`no_latent_space`	Classification directe sans goulot ($768 \to 21$)	27 836 277	**82,16%**	50,06%	96,05%	0,6153	5 (Early stop)	15m 50s
`frozen_backbone`	Fine-tuning désactivé (Backbone ConvNeXt gelé)	405 525	79,60%	44,73%	94,69%	0,8059	10	16m 26s

les premiers essais avec un batch size standard de 32 ou des résolutions de 384×384 ont immédiatement provoqué des erreurs critiques `CUDA Out of Memory`. - **Solutions apportées** : 1. Fixation du batch size à 16, calibré pour occuper ~3,4 Go de VRAM. 2. Activation systématique de la précision mixte avec `torch.autocast(device_type='cuda', dtype=torch.float16)` et mise à l'échelle des gradients via `torch.GradScaler()`. 3. Libération explicite des tenseurs et invocation de `torch.cuda.empty_cache()` et `gc.collect()` entre chaque pli de validation croisée et chaque exécution d'ablation.

2. Bruit taxonomique et incohérences du dataset ART500K

Le dataset brut comporte des libellés multiples, des doublons historiques et des dénominations ambiguës (*Ashcan School*, *Barbizon school*, *Modernism*). Certaines œuvres étaient associées à plusieurs dossiers simultanément. - **Solutions apportées** : 1. Conception de `src/loader.py` avec une résolution stricte de priorité excluant les dénominations trop génériques (*Art Moderne*, *Réalisme Social*) lorsqu'une étiquette plus spécifique existe. 2. Validation de la démarche par la comparaison systématique des matrices de confusion canonique et non-canonique.

3. Volume de données et compromis sur les sous-ensembles

ART500K représente plus de 56 Go et ~500 000 œuvres. Le dossier `Artists1` manquait presque totalement d'étiquetage de mouvements artistiques, tandis qu'`Artists2` introduisait un déséquilibre massif d'artistes majeurs au détriment des courants picturaux minoritaires. - **Solution apportée** : Focalisation ciblée sur le dossier `Art Movement` (16 612 œuvres vérifiées), permettant de maîtriser les temps d'expérimentation (campagnes d'ablation complètes en ~1h30 au lieu de dizaines d'heures) tout en conservant une diversité stylistique équilibrée. L'intégration progressive d'Artists2 est formalisée pour la version V2.

4. Déploiement du protocole MCP et gestion des flux asynchrones

L'interfaçage du modèle de vision avec le serveur MCP a nécessité la gestion d'entrées hétérogènes (fichiers locaux, URLs Web distantes, identifiants d'œuvres). - **Solutions apportées** : Création d'un module d'assistance (`src/serv_mcp/utils.py`) avec téléchargement et mise en cache MD5 des images Web, validation de format avec Pillow, et pré-calcul d'un index matriciel (`gallery_index.pt`) pour garantir un temps de réponse instantané lors des appels d'outils par le LLM.

IV. Conclusion & Perspectives

Compétences acquises

Ce projet nous a permis de franchir un cap significatif en Deep Learning appliqué à la vision et aux architectures agentiques : 1. **Maîtrise du Transfer Learning & Fine-tuning** : Adaptation réussie d'un réseau convolutif de pointe (`ConvNeXt-Tiny`) aux spécificités plastiques et textuelles des œuvres d'art. 2. **Rigueur méthodologique et expérimentale** : Implémenta-

tion d'une suite complète de validation croisée stratifiée, d'études d'ablation quantifiées et de matrices de confusion normalisées. 3. **Architecture Agentique & Protocole MCP** : Conception d'un pont fonctionnel entre l'intelligence conversationnelle des LLM et la précision d'un modèle spécialiste de vision par ordinateur.

Limites actuelles : Analyse du partitionnement des données

Une analyse critique de notre implémentation actuelle (`src/loader.py`) révèle que le jeu de données est partitionné en **deux ensembles** : **Train (80%)** et **Validation (20%)**. - Bien que la validation croisée stratifiée à 3 plis apporte une garantie de reproductibilité, les métriques finales et les matrices de confusion ont été évaluées sur l'ensemble de validation qui a également guidé l'Early Stopping. - **Recommandation pour la version de production** : Il convient d'introduire un découpage strict à **trois volets indépendants** : **Entraînement (70%)**, **Validation (15%)** (dédié exclusivement au calcul de la loss et à l'arrêt anticipé), et **Test (15%)** (isolé hermétiquement, évalué une seule fois à la fin).

Pistes d'amélioration futures (Version 2)

1. **Intégration d'Artists2** : Intégrer les œuvres issues d'Artists2 après rééquilibrage par échantillonnage stratifié ou pondération de loss (*Class-Balanced Focal Loss*) pour enrichir les représentations d'artistes emblématiques.
2. **Apprentissage métrique / Contrastive Learning** : Entraîner l'espace latent 512D avec une perte de triplet (*Triplet Margin Loss*) ou une perte Supervised Contrastive (SupCon). Cela permettrait de rapprocher explicitement les œuvres d'un même mouvement et d'éloigner les styles divergents, décuplant l'efficacité de l'outil MCP `find_similar_artworks`.
3. **Modèles Multimodaux de bout en bout (Vision-Language)** : Enrichir le modèle spécialiste en intégrant les légendes textuelles et notices d'œuvres pour un alignement conjoint texte-image (type CLIP adapté à l'histoire de l'art).

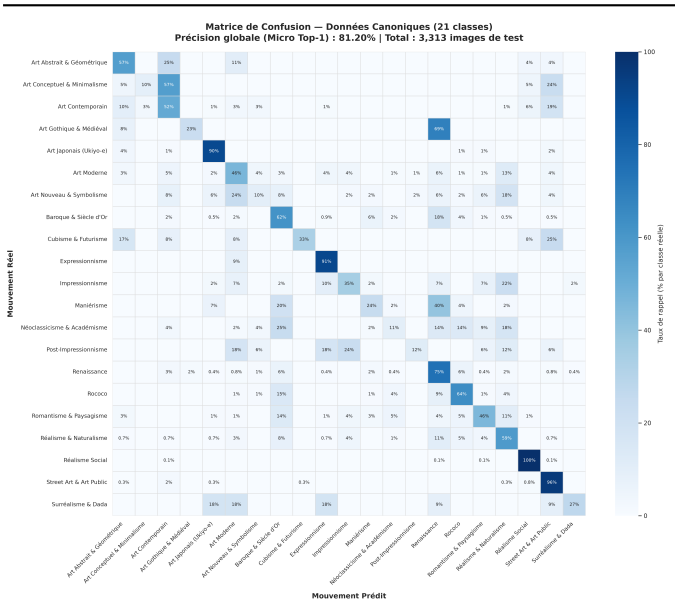
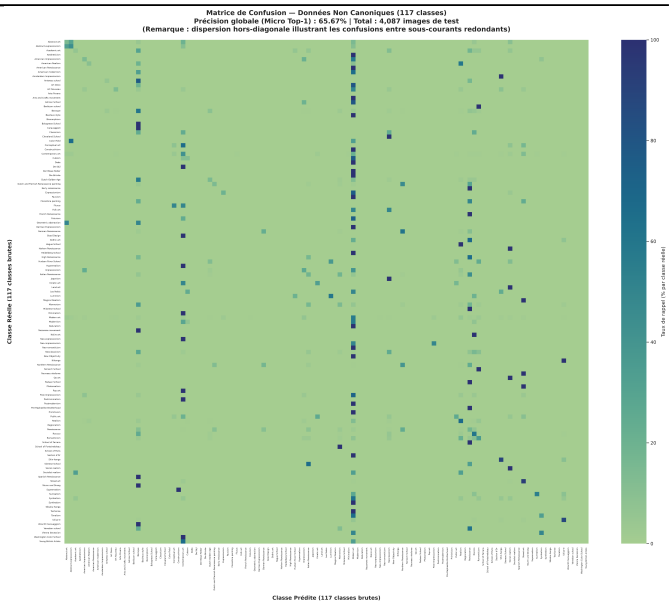
V. Annexes

Annexe A : Visualisation comparative des matrices de confusion

Annexe B : Configuration client pour le serveur MCP

Pour activer le serveur dans Claude Desktop, Cursor ou un client compatible, insérer la configuration suivante dans le fichier `mcpServers` :

```
{
  "mcpServers": {
    "art-explorer": {
      "command": "/chemin/absolu/vers/project/.venv/bin/python3",
      "args": ["-m", "src.serv_mcp.mcp_server"],
      "cwd": "/chemin/absolu/vers/project"
    }
  }
}
```

Matrice Canonique (21 classes)	Matrice Non Canonique (117 classes)
	
Micro Top-1 : 81,20% — Structure diagonale nette	**Micro Top-1 : 65,67%** — Forte dispersion hors-diagonale